

算法设计与分析作业 - 第三章

3-1 最长单调递增子序列的 $O(n^2)$ 算法

这道题的意思是在一串乱序的数字里，找出一串从小到大排好序的子序列，看谁最长。如果我们要知道以第 i 个数结尾的序列最长是多少，我得先看它前面那些比它小的数。比如前面有个数 $A[j]$ 比当前这个 $A[i]$ 小，那 $A[i]$ 就可以直接跟在 $A[j]$ 后面，长度就是在原来的基础上加 1。所以我只需要把前面所有比 $A[i]$ 小的情况都扫一遍，挑个最长的接上去就行。

【解题过程】

1. 设一个数组 $dp[i]$ ，代表以第 i 个数字结尾时，能凑出的最长递增序列的长度。
2. 算 $dp[i]$ 的时候，我回头看 $j = 1$ 到 $i - 1$ 这些位置。只要发现 $A[j] < A[i]$ ，我就尝试更新： $dp[i] = \max(dp[i], dp[j] + 1)$ 。当然，初始的时候每个数自己都能当一个序列，所以 dp 全都初始化为 1。
3. 把整个 dp 数组填满后，里面最大的那个值就是我们要的答案。

【参考代码】

```
1  int lengthOfLIS(vector<int>& nums) {
2      int n = nums.size();
3      vector<int> dp(n, 1);
4      int ans = 1;
5      for (int i = 0; i < n; i++) {
6          for (int j = 0; j < i; j++) {
7              if (nums[j] < nums[i])
8                  dp[i] = max(dp[i], dp[j] + 1);
9          }
10         ans = max(ans, dp[i]);
11     }
12     return ans;
13 }
```

【复杂度】

因为算每一个数都要把前面所有的数看一遍，两层循环嵌套，所以时间复杂度是 $O(n^2)$ 。

3-2 最长单调递增子序列的 $O(n \log n)$ 算法

我们用一个数组 B 来存不同长度的递增子序列的“最小末尾数”。比如长度为 3 的子序列，末尾是 5 肯定比末尾是 10 好，因为 5 后面更容易接新的数。

这样我们在遍历新数字时，如果它很大，就直接把序列拉长；如果它不够大，就用它去替换掉名单里第一个比它大的数，把那个位置的末尾数值压低，给后面留机会。

【解题过程】

1. 建立一个辅助数组 B 。遍历原序列中的每个数 x 。
2. 如果 x 比 B 里面最后一个数还大，那太好了，直接 **append** 进去，最长长度加 1。
3. 如果 x 没那么大，我就在 B 里面找。因为 B 是从小到大排好序的，我可以用二分查找快速定位到第一个 $\geq x$ 的位置，然后用 x 替换它。
4. 最后 B 数组的长度就是最长子序列的长度。

【参考代码】

```
1 int lengthOfLIS(vector<int>& nums) {
2     vector<int> B;
3     for (int x : nums) {
4         auto it = lower_bound(B.begin(), B.end(), x);
5         if (it == B.end()) B.push_back(x);
6         else *it = x;
7     }
8     return B.size();
9 }
```

【复杂度】

二分查找是 $\log n$ ，外面套个 n 次循环，总共就是 $O(n \log n)$ 。

3-4 整数线性规划问题的动态规划算法

这其实就是“完全背包”。 b 是背包总重， a_i 是每个东西的重量， c_i 是价值。问你怎么装能让价值 $c_i x_i$ 的总和最大。这里的 x_i 是非负整数，意思就是每种东西想拿多少个都行，只要不把背包装爆 ($\leq b$)。

【解题过程】

1. 设 $f[j]$ 为背包容量为 j 时能装下的最大价值。
2. 既然东西可以无限拿，那对于容量 j ，尝试每一种物品。如果拿了一个物品 i ，那剩下的空间就是 $j - a_i$ ，这时候的最大价值就是 $f[j - a_i] + c_i$ 。
3. 方程： $f[j] = \max(f[j], f[j - a_i] + c_i)$ 。
4. 咱们从容量 1 一直算到 b ，每次算 $f[j]$ 都把 n 种物品试一遍。

【参考代码】

```
1  int unboundedKnapsack(int b, vector<int>& a, vector<int>& c) {
2      vector<int> f(b + 1, 0);
3      for (int j = 1; j <= b; j++) {
4          for (int i = 0; i < a.size(); i++) {
5              if (j >= a[i])
6                  f[j] = max(f[j], f[j - a[i]] + c[i]);
7          }
8      }
9      return f[b];
10 }
```

【复杂度】

容量 b 乘上物品数 n ，复杂度是 $O(nb)$ 。

3-5 二维约束的 0-1 背包问题

这题比普通的背包多了一个限制。以前只是不能超重，现在不仅不能超重（重量 w ），还不能太占地方（体积 b ）。而且它是 0-1 背包，意味着每件东西要么拿一个，要么不拿，不能拆开拿。

【解题过程】

1. 现在得用二维数组 $dp[j][k]$ ，代表重量限制为 j 、体积限制为 k 时的最大价值。
2. 遍历到第 i 个物品时：
 - 如果不拿它，价值还是原来的 $dp[j][k]$ 。
 - 如果拿它，价值就变成 $dp[j - w_i][k - b_i] + v_i$ 。
 - 在两个里面挑个大的。
3. 写代码时，为了省空间不用三维数组，我们要把重量和体积这两层循环都从大往小写。这是为了保证我们在算当前物品时，参考的还是“还没放这件物品”之前的旧数据，防止一件东西被放进去好几次。

【参考代码】

```
1  int knapsack2D(vector<int>& w, vector<int>& b, vector<int>& v,  
    int C, int D) {  
2      vector<vector<int>> dp(C + 1, vector<int>(D + 1, 0));  
3      for (int i = 0; i < w.size(); i++) {  
4          for (int j = C; j >= w[i]; j--) {  
5              for (int k = D; k >= b[i]; k--) {  
6                  dp[j][k] = max(dp[j][k],  
7                      dp[j - w[i]][k - b[i]] + v[i]);  
8              }  
9          }  
10     }  
11     return dp[C][D];  
12 }
```

【复杂度】

时间复杂度是 $O(n \cdot C \cdot D)$ ，就是物品数、载重和容积三个数连乘。虽然算起来慢一点，但在动态规划里这是最稳的解法。